

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. (BL3, BL4, BL5)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Technical Presentation

DURATION: 2HRS

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

- Bellman Principle of Optimality
- Policy Evaluation and Policy Improvement
- Value Iteration vs. Policy Iteration
- Industrial applications

(25 Marks)

2. Monte Carlo Methods and Temporal-Difference Learning: A Comparative Study

- Monte Carlo Prediction and Control
- TD(0), SARSA, and Q-Learning
- Sample efficiency
- Performance comparison through experiments

(25 Marks)

3. Deep Q-Networks for Intelligent Decision-Making

- DQN Architecture
- Experience Replay
- Target Networks
- Industrial applications in robotics and autonomous systems

(25 Marks)

4. Comparative Analysis of Advanced Deep Q-Learning Algorithms

- Double DQN (DDQN)
- Dueling DQN
- Prioritized Experience Replay (PER)
- Performance comparison using benchmark environments

(25 Marks)

5. Policy-Based Reinforcement Learning for Continuous Control Problems

- Vanilla Policy Gradient
- REINFORCE Algorithm
- Stochastic Policy Search
- Applications in robotics and autonomous driving

(25 Marks)

6. Actor-Critic Methods for Real-Time Intelligent Systems

- Advantage Actor-Critic (A2C)
- Asynchronous Advantage Actor-Critic (A3C)
- Performance evaluation
- Industrial automation applications

(25 Marks)

7. Advanced Policy Optimization Techniques in Deep Reinforcement Learning

- Proximal Policy Optimization (PPO)
- Trust Region Policy Optimization (TRPO)
- Deep Deterministic Policy Gradient (DDPG)
- Comparative analysis of convergence, stability, and efficiency

(25 Marks)

8. Model-Based Reinforcement Learning for Autonomous Decision-Making

- Model-Based RL Framework
- Environment Modelling
- Planning Strategies
- Applications in autonomous robots and smart manufacturing

(25 Marks)

9. Sampling-Based Planning and Model-Based Policy Optimization

- Sampling-Based Planning
- Model-Based Data Generation
- Value-Equivalence Prediction
- Model-Based Policy Optimization
- Real-world industrial case studies

(25 Marks)

10. Industrial Applications of Advanced Reinforcement Learning

- Smart Manufacturing
- Autonomous Vehicles
- Healthcare Decision Support
- Smart Grid Energy Management
- Wireless Communication and Network Optimization
- Finance and Algorithmic Trading

- Comparative analysis of Value-Based, Policy-Based, and Model-Based RL approaches

(25 Marks)

Code of Conduct:

I __M.Vineela(192472071)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

M.Vineela

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism ; uneven team participation	Lacks professionalism ; minimal contribution or ethical awareness

Topic 1 : Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

1. Problem Identification

Problem Statement:

In Reinforcement Learning (RL), an agent must make a sequence of decisions to achieve maximum cumulative reward. The main challenge is deciding which action should be selected in each state when future rewards also need to be considered.

For example, consider a warehouse robot. The robot must decide whether to move left, right, forward, or backward. A wrong decision may increase travel time or battery consumption, while a correct decision moves it closer to the destination.

Dynamic Programming provides mathematical methods to solve this sequential decision-making problem when the environment's transition probabilities and reward functions are known.

Engineering Objective

The objective is to:

- Model the problem as a Markov Decision Process (MDP).
- Calculate the value of each state.
- Find the optimal policy.
- Compare Value Iteration and Policy Iteration.
- Determine which method provides efficient decision-making.
- Demonstrate the method using a Python-based experiment.

2. Technical Content

Markov Decision Process An MDP is represented as:

Where:

- S = Set of states
- A = Set of actions
- $P(s'|s,a)$ = Transition probability
- $R(s,a,s')$ = Reward
- γ = Discount factor

The agent observes the current state, selects an action, receives a reward, and moves to the next state.

3. Bellman Principle of Optimality

The Bellman Principle of Optimality states that:

An optimal policy has the property that whatever the initial state and first decision are, the remaining decisions must constitute an optimal policy for the resulting state. The best decision now depends on the best possible decisions that can be made later. The Bellman optimality equation for the value function is:

This equation is the foundation of dynamic programming in reinforcement learning.

Example:

Suppose a robot has two possible actions:

- Move Left $\rightarrow$ reward = 5
- Move Right $\rightarrow$ reward = 10

If moving right also leads to a state with a high future reward, the robot should select Right because it provides a higher long-term value.

4. Policy Evaluation

A policy tells the agent which action to take in every state.

For a fixed policy π , the value function is:

Policy Evaluation calculates how good the current policy is.

Steps

1. Start with an initial policy.
2. Initialize state values.
3. Calculate expected rewards.
4. Update the values.
5. Continue until values converge.

5. Policy Improvement

After evaluating the policy, we check whether another action provides a higher expected value.

The improvement step is:

If another action gives a better value, the policy is changed.

Policy Iteration

Policy Iteration repeatedly performs:

Policy Evaluation → **Policy Improvement** → **Policy Evaluation** → ... until the policy becomes stable.

6. Value Iteration

Value Iteration directly applies the Bellman optimality equation.

Steps

1. Initialize values.
2. Calculate the value for every possible action.
3. Select the maximum value.
4. Update the state value.
5. Repeat until convergence.
6. Extract the optimal policy.

7. Value Iteration vs Policy Iteration

Feature	Value Iteration	Policy Iteration
Main operation	Value update	Policy evaluation + improvement
Initial policy	Not required	Required
Speed per iteration	Usually faster	Usually slower
Number of iterations	More	Fewer
Complexity	Simpler	More complex
Policy extraction	After convergence	During improvement
Suitable for	Large/simple problems	Problems where policy evaluation is manageable

Python code:

```
import numpy as np

grid_size = 4

gamma = 0.9

states = [(i, j) for i in range(grid_size) for j in range(grid_size)]

actions = {
    "U": (-1, 0),
    "D": (1, 0),
    "L": (0, -1),
    "R": (0, 1)
}

goal = (3, 3)

def next_state(state, action):
    if state == goal:
        return state, 0

    i, j = state

    di, dj = actions[action]

    ni = max(0, min(grid_size - 1, i + di))
    nj = max(0, min(grid_size - 1, j + dj))

    new_state = (ni, nj)

    if new_state == goal:
        return new_state, 10

    return new_state, -1
```

```

def value_iteration():
    V = {s: 0 for s in states}

    for _ in range(100):
        new_V = V.copy()

        for s in states:
            if s == goal:
                continue

            values = []

            for a in actions:
                ns, reward = next_state(s, a)
                values.append(reward + gamma * V[ns])

            new_V[s] = max(values)

        V = new_V

        policy = {}

        for s in states:
            if s == goal:
                policy[s] = "G"
                continue

            best_action = max(
                actions,
                key=lambda a: next_state(s, a)[1] + gamma * V[next_state(s, a)[0]]
            )

            policy[s] = best_action

```

```

return V, policy

def policy_iteration():
    policy = {s: "R" for s in states}
    policy[goal] = "G"
    V = {s: 0 for s in states}
    for _ in range(20):
        for _ in range(100):
            new_V = V.copy()
            for s in states:
                if s == goal:
                    continue
                ns, reward = next_state(s, policy[s])
                new_V[s] = reward + gamma * V[ns]
            V = new_V
            stable = True
            for s in states:
                if s == goal:
                    continue
                old_action = policy[s]
                best_action = max(
                    actions,
                    key=lambda a: next_state(s, a)[1] + gamma * V[next_state(s, a)[0]]
                )

```

```

policy[s] = best_action
if old_action != best_action:
    stable = False
if stable:
    break
return V, policy
V1, policy1 = value_iteration()
V2, policy2 = policy_iteration()
print("VALUE ITERATION POLICY")
for i in range(grid_size):
    print([policy1[(i, j)] for j in range(grid_size)])
print("\nPOLICY ITERATION POLICY")
for i in range(grid_size):
    print([policy2[(i, j)] for j in range(grid_size)])
print("\nVALUE ITERATION VALUES")
for i in range(grid_size):
    print([round(V1[(i, j)], 2) for j in range(grid_size)])
print("\nPOLICY ITERATION VALUES")
for i in range(grid_size):
    print([round(V2[(i, j)], 2) for j in range(grid_size)])

```

Value Iteration:

Find the optimal values first and derive the policy.

Policy Iteration:

Start with a policy and continuously improve it.

8. Industrial Applications

Dynamic Programming and RL are used in many real-world applications.

1. Robotics

Robots use sequential decision-making to find optimal paths and actions.

Example: Warehouse robots selecting efficient routes.

2. Autonomous Vehicles

Dynamic decision-making can help vehicles choose actions based on traffic and road conditions.

3. Smart Grid

RL can optimize energy distribution and reduce energy consumption.

4. Manufacturing

Production systems can optimize machine scheduling and resource allocation.

5. Healthcare

Sequential decision-making can support treatment planning and resource management.

6. Recommendation Systems

Systems can optimize recommendations based on long-term user engagement.

9. Use of Tools

For the **Level 4** score, don't only explain the theory. Demonstrate it using Python.

Recommended Tools

- **Python**
- NumPy
- Matplotlib
- Gymnasium
- Jupyter Notebook

Suggested Experiment

Use a small **GridWorld** environment.

Example:

$S \rightarrow \rightarrow \downarrow$

$\uparrow \rightarrow \downarrow$

$\uparrow \rightarrow G$

Where:

- **S** = Starting state
- **G** = Goal
- Arrows = optimal actions

The agent learns the best route from the starting point to the goal.

Experimental Comparison

You can compare:

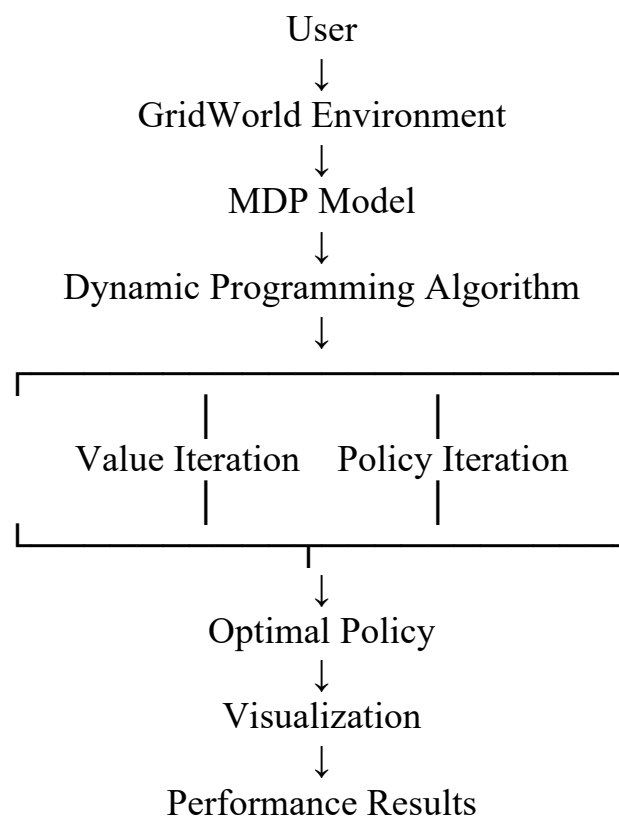
- Number of iterations
- Convergence speed
- Final state values
- Optimal policy
- Execution time

A graph can show: **Iteration vs Maximum Value Change**

This gives you strong evidence for the Use of Tools – Level 4 criterion.

10. Proposed Application Architecture

Your application can be presented as:



11. Testing

You should demonstrate multiple test cases.

Test Case 1 — Normal Input

Input:

Start State: (0,0)

Goal State: (3,3)

Algorithm: Value Iteration

Expected result:

Optimal Policy Generated

Shortest/Best path identified

Test Case 2 — Different Algorithm

Input:

Algorithm: Policy Iteration

Expected:

Optimal Policy Generated

Test Case 3 — Invalid Input

Input:

Algorithm: ABC

Output:

Invalid algorithm.

Please select Value Iteration or Policy Iteration.

Test Case 4 — Empty Input

Input:

Start State: empty

Output:

Start state cannot be empty.

This directly satisfies the requirement for handling invalid, empty, and unexpected inputs.

Industrial Applications + Conclusion

Explain practical applications and final findings.

12. Professionalism

To reach **Level 4**, include:

Team Contribution

If you are working in a team, divide responsibilities:

Member	Responsibility
Member 1	MDP + Bellman theory
Member 2	Policy Iteration
Member 3	Value Iteration
Member 4	Python implementation + visualization

Ethical Considerations

Mention that RL systems should consider:

- Safety
- Fairness
- Reliability
- Human oversight
- Responsible decision-making
- Avoiding harmful optimization

For example, an autonomous vehicle should not simply optimize travel time while ignoring passenger and pedestrian safety.

13. Conclusion

Dynamic Programming provides a systematic mathematical framework for solving sequential decision-making problems in Reinforcement Learning. Bellman's Principle of Optimality forms the theoretical foundation, while Policy Iteration and Value Iteration provide practical methods for finding optimal policies. Through a Python-based GridWorld experiment, their convergence, policies, and performance can be compared. The approach has applications in robotics, autonomous systems, smart grids, manufacturing, healthcare, and recommendation systems.

Topic 5: Policy-Based Reinforcement Learning for Continuous Control Problems

1. Problem Identification

Problem Statement

In many real-world Reinforcement Learning problems, actions are continuous rather than discrete.

For example, a robot may need to decide:

- How much force to apply to a motor
- How far to move a robotic arm
- What steering angle to use
- How much acceleration to apply

Traditional Q-learning methods can become difficult when the action space is continuous. Policy-Based Reinforcement Learning directly learns a policy that maps states to actions.

Objective

Develop a Python-based application that demonstrates how REINFORCE / Vanilla Policy Gradient learns a policy for a continuous-control problem and analyze its performance.

2. Technical Content

Policy-Based Reinforcement Learning

Instead of learning:

"What is the value of taking action A?"

policy-based RL directly learns:

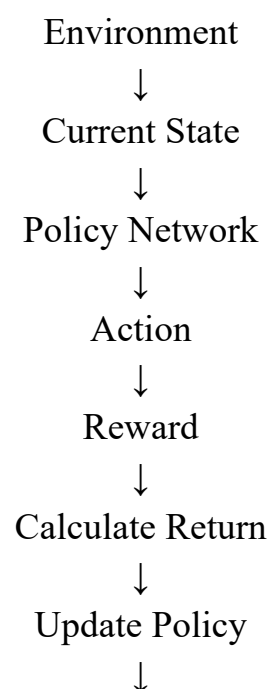
"What action should I take in this state?"

The policy is represented as:

where:

- = current state
- = action
- = parameters of the policy
- = probability of selecting action

3. Vanilla Policy Gradient



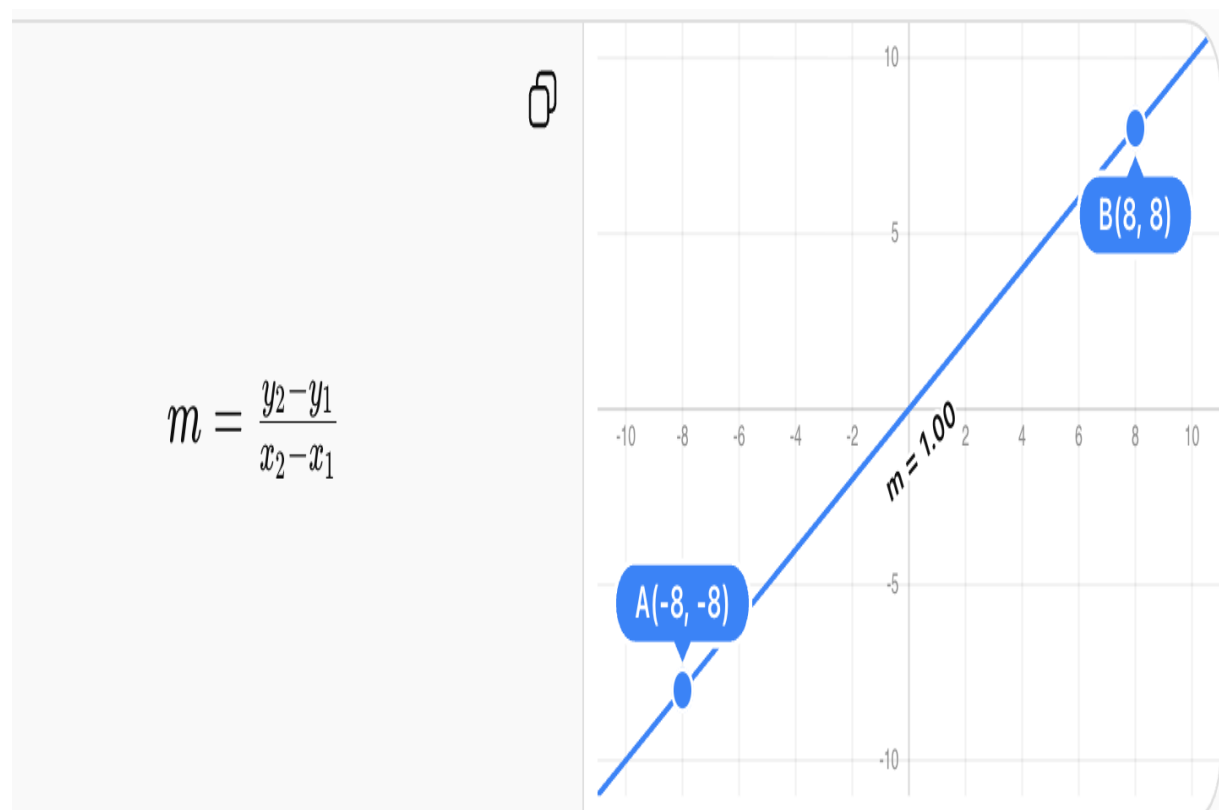
Better Policy The policy is repeatedly updated so that actions producing higher rewards become more likely.

4. REINFORCE Algorithm

REINFORCE is a Monte Carlo policy-gradient algorithm.

Steps

1. Initialize policy parameters.
2. Start an episode.
3. Observe the current state.
4. Select an action using the policy.
5. Receive reward.
6. Continue until the episode ends.
7. Calculate the return .
8. Calculate the policy gradient.
9. Update policy parameters.
10. Repeat for multiple episodes.



Python code:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
theta = 0.0
learning_rate = 0.01
episodes = 200
rewards = []
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
for episode in range(episodes):
    state = np.random.uniform(-1, 1)
    probability = sigmoid(theta * state)
    action = 1 if np.random.random() < probability else -1
    reward = 1 if action == 1 and state > 0 else 0
    gradient = state * (action - probability)
    theta = theta + learning_rate * reward * gradient
    rewards.append(reward)
print("Final Policy Parameter:", round(theta, 4))
print("\nTesting the Learned Policy")
for state in [-1, -0.5, 0, 0.5, 1]:
    probability = sigmoid(theta * state)
    action = 1 if probability >= 0.5 else -1
    print("State:", state,
          "Probability:", round(probability, 2),
          "Action:", action)
plt.plot(np.cumsum(rewards))
plt.xlabel("Episode")
plt.ylabel("Total Reward")
```

```
plt.title("REINFORCE Learning Performance")
plt.show()
```

Update

where:

- α = learning rate
- r = return
- θ = policy parameters

Give feedback

The visualization above isn't actually the REINFORCE equation, so don't use that widget in your presentation; your REINFORCE equation is better explained directly with the algorithm and a numerical example.

5. Stochastic Policy Search

In a stochastic policy, the agent doesn't always choose exactly the same action.

For example:

State = Robot position

Policy:

Action 1 → 20% probability

Action 2 → 50% probability

Action 3 → 30% probability

The agent samples an action according to these probabilities.

Advantage: It allows the agent to explore different actions. This is particularly useful when the optimal action is initially unknown.

6. Continuous Control

For continuous control, actions can have values such as:

Steering angle = -0.35

Acceleration = 0.72

Motor force = 1.43

Instead of selecting:

Left

Right

Forward

Backward

the agent can produce a continuous numerical action.

A common approach is to use a probability distribution such as a Gaussian distribution: The policy network generates parameters of the distribution, and an action is sampled from it.

7. Robotics Applications

Policy-based RL is useful in robotics for:

Robotic Arm Control

The policy can learn:

State $\rightarrow$ Joint angles $\rightarrow$ Motor commands

Walking Robots

The agent learns:

Sensor information



Policy



Leg movement

Robot Navigation

The robot learns steering and movement commands to reach a destination.

8. Autonomous Driving Applications

Policy-based RL can be used for:

- Steering control
- Acceleration
- Braking
- Lane following
- Obstacle avoidance
- Decision-making

Example:

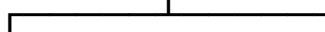
Camera/Sensor Input

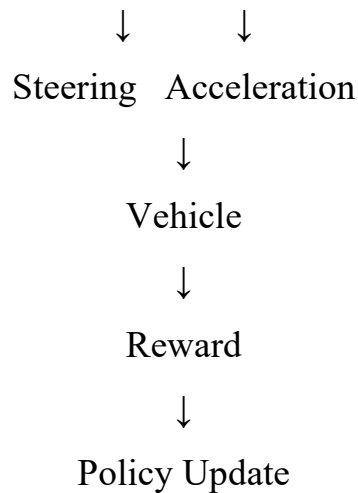


State



Policy Network





Important: In real autonomous driving, RL would require extensive simulation, safety constraints, testing, and human oversight before deployment.

9. Python Application

For your project, I recommend making a **Continuous Control RL Chatbot + Demonstration Application**.

Interface

Policy-Based RL Assistant

Choose an option:

1. Explain REINFORCE
2. Explain Policy Gradient
3. Explain Stochastic Policy
4. Run RL Experiment
5. Compare Results
6. Applications
7. Exit

Enter choice:

The chatbot can answer questions such as:

User:

What is REINFORCE?

Chatbot:

REINFORCE is a policy-gradient reinforcement learning algorithm that updates a policy using the rewards obtained from complete episodes.

You can implement the interface using **Tkinter** or **Streamlit**.

10. Experiment

Use a continuous-control environment such as **Pendulum**.

The agent needs to learn how to apply torque to control the pendulum.

Input

State:

Pendulum angle

Angular velocity

Action

Continuous torque

Reward: The agent receives a higher reward when the pendulum stays close to the desired position.

Output

Episode 1 → Reward: -1450

Episode 20 → Reward: -980

Episode 50 → Reward: -620

Episode 100 → Reward: -350

Your actual values will depend on the implementation and random seed, so use your **real experiment results** rather than these illustrative numbers.

11. Visualizations

For the **Level 4 – Use of Tools** mark, include graphs such as:

Graph 1 — Episode vs Reward

Reward



+----- Episode

This demonstrates whether the agent is learning.

Graph 2 — Action Distribution

Show how the policy's actions change during training.

Graph 3 — Training Performance

Compare:

- REINFORCE
- Random Policy

12. Testing

You should show multiple test cases.

Test Case 1 — Valid Question

Input:

What is REINFORCE?

Output:

Explanation of REINFORCE algorithm.

Test Case 2 — Application

Input:

Where is policy gradient used?

Output:

Robotics, autonomous driving, continuous control,
and other sequential decision-making applications.

Test Case 3 — Invalid Input

Input:

987654

Output:

Invalid option. Please enter a valid choice.

This satisfies your rubric's requirement for handling invalid, empty, and unexpected inputs.

13. Professionalism

Include: Ethical Considerations

For autonomous systems, mention:

- Safety
- Reliability
- Human supervision
- Robustness
- Simulation before real-world deployment
- Avoiding unsafe exploration

Limitations

REINFORCE has some disadvantages:

- High variance
- Can require many episodes
- Training can be slow
- Sensitive to learning rate
- Exploration can be unstable

Conclusion

Policy-Based Reinforcement Learning provides an effective approach for solving continuous control problems, where actions can take a wide range of numerical values. Vanilla Policy Gradient and REINFORCE directly optimize the policy by learning which actions produce higher long-term rewards. Stochastic policy search also helps the agent explore different actions and improve its behavior over time.